

Where it breaks

An honest attempt to break my own site — <https://yuguda.netlify.app> — rather than walk the happy path.
Method: direct POST s that bypass browser validation, every link checked including externals, path and casing probes, oversized and unicode payloads, header and caching inspection, and timing runs.

Fixed now

1. No custom 404 page

Found: any wrong URL (/nonexistent , /a/b/c/d) served Netlify's generic branded "Page not found" screen. A recruiter following a stale link would land on a page that looks like the site doesn't exist. **Fixed:** 404.html in the site's own design, with routes back to the home page and the contact section, plus my email. Netlify serves a root 404.html automatically.

2. Site was framable by any origin — no security headers

Found: no X-Frame-Options , X-Content-Type-Options , Referrer-Policy , or Permissions-Policy . Any site could iframe mine and overlay it (clickjacking), and browsers were free to MIME-sniff responses. **Fixed:** _headers file setting all four: X-Frame-Options: DENY , nosniff , strict-origin-when-cross-origin , and a Permissions-Policy denying geolocation/mic/camera.

3. Contact form couldn't send a second message

Found: reading my own submit handler — after a successful send, the button was disabled and relabelled "Sent" permanently. Anyone wanting to send a follow-up had to reload the page. The disable is correct (it blocks double-tap double-submits); leaving it disabled forever was not. **Fixed:** re-enables after 4 seconds as "Send another message" — still long enough to swallow an accidental double-tap.

4. Every asset revalidated on every visit

Found: all static assets served cache-control: public, max-age=0, must-revalidate . Repeat visitors pay a round-trip per image just to be told nothing changed. **Fixed:** one-day cache for /assets/* , images, SVG, and the CV. Not longer, because filenames aren't content-hashed — an updated screenshot would otherwise stay stale.

5. Contact address didn't match where mail actually landed

Found by review, and it wasn't on my list at all. Every visible address on the site — hero CTA, form fallback, footer, thank-you page, 404 page, and the error message in the form's own JavaScript — advertised muhammed.yuguda@adeptengr.com . But form submissions were configured to notify elemenx93@gmail.com , and the Calendly link sits under the same elemenx93 handle.

So the site asked recruiters to contact one identity while delivering to another. A reply would arrive from an address that doesn't appear anywhere on the site, and mail sent to the advertised address depended on my monitoring an inbox I don't actually read first. **Fixed:** consolidated on elemenx93@gmail.com across all seven occurrences in three files, so the advertised address, the form destination, and the booking handle are one identity. An address I genuinely monitor beats a better-looking one I don't.

Known limitations — named, not hidden

A. No server-side validation on the contact form

A direct POST with every field empty is accepted (HTTP 200). So is a malformed address like not-an-email . The required and type="email" attributes only bind a real browser; curl ignores them entirely.
Why it stays: the site is static, and Netlify Forms on the free tier has no validation hook. Fixing it properly means a Netlify Function, which is more moving parts than a personal contact form warrants.
What limits the damage: the honeypot field (bot-field) catches automated submissions, and a junk entry costs me one ignored email rather than corrupting anything. Nothing submitted is ever rendered back onto the page, so there is no stored-XSS path.

B. Rapid repeated submissions are rate-limited — now measured

My first write-up called this an unresolved observation: submissions started failing after a burst and hadn't recovered "after several minutes." My reviewer pushed back on exactly that, and he was right — "that's not a known limitation, that's an open incident." A thing you haven't characterised isn't a tradeoff you've accepted. So I measured it.

	Result
Threshold	~10 rapid submissions from one IP
Response	HTTP 429 Too Many Requests — the correct status, not an error page
Recovery	~4 minutes, confirmed by polling until it returned 200

That converts it to a documented limitation: Netlify rate-limits form submissions per IP, responds with a standards-compliant 429, and clears within about four minutes. A visitor would have to submit ten times in seconds to trigger it, and my handler surfaces a fallback ("Something broke sending that. Email me directly: ...") rather than a dead end.

The separate bug hiding underneath it. The original persistent failures returned **404**, not 429 — a different thing entirely. Those 404s happened *before* the hardening deploy and stopped after it. Netlify registers forms by parsing HTML at deploy time, so form registration had got into a bad state and a redeploy re-registered it. That's the actionable finding: **if the form 404s, redeploy — a 429 is throttling and clears itself**. I would have filed the whole thing under "abuse protection" and been wrong if I hadn't gone back and measured.

C. `mailto:` still fails silently for some visitors

The form exists precisely because `mailto:` does nothing on a machine with no mail client configured. The address is still shown as a fallback in three places, but for a visitor with neither a working mail client nor a working form, there is no path left. Accepted risk; the alternative is a second provider.

D. LinkedIn returns HTTP 999 to automated checks

My link checker reports 999 for `linkedin.com/in/yuguda`. That is LinkedIn's anti-bot response, not a broken link — it resolves normally in a browser. Recorded so a future check doesn't chase it.

E. Video is 1.9 MB

Compressed from 52 MB, and it loads `preload="metadata"` so nothing but the header downloads until a visitor presses play. Range requests work (HTTP 206), so seeking is fine. Still a real cost on a metered mobile connection — but the alternative is not showing the agent actually running.

Findability and speed

SEO / meta added: page title, meta description, canonical URL, full Open Graph and Twitter card tags with a generated 1200×630 preview image, favicons (SVG plus PNG fallbacks and an apple-touch-icon), a web manifest, `robots.txt`, and `sitemap.xml`. The thank-you and 404 pages are marked `noindex` — confirmation screens have no business in search results.

Speed, measured live:

Metric	Result
TTFB	0.86 – 1.27 s
Total load (HTML)	1.26 – 1.63 s
HTML transferred	9.9 KB brotli, from 38 KB raw (74% saved)
Full first paint payload	~390 KB including three screenshots
Compression	Brotli, automatic
Video on first load	0 bytes (<code>preload="metadata"</code>)

Heaviest single asset is the vLLM screenshot at 266 KB, already downsampled to 1400px and quantized from 498 KB.

Findability caveat: the site is new, so searching my name does not surface it yet. `sitemap.xml` and `robots.txt` are in place; indexing takes time and is not something a deploy can force.

Hardening review

Reviewed by Fuhad, backend developer, after I'd done my own pass.

He rejected most of my triage as already sound, and pushed on one thing: item B. My write-up had it as observed-and-unresolved — I didn't know whether it recovered, and had said so. His point: *"that's not a known limitation, that's an open incident,"* and a recruiter double-clicking impatiently on a slow connection could be the one to find it live, with no way for me to know they'd hit a wall. He wanted it converted to one of two states before launch: confirmed-recovers-after-X, or actually broken.

I measured it. It recovers in ~4 minutes, returns a correct 429, and needs ~10 rapid submissions to trigger — now documented as limitation B. Doing that also surfaced a genuine bug I'd misfiled: the original persistent failures were 404s from stale form registration, fixed by redeploying, not the same thing as throttling at all.

His one new must-fix wasn't on my list: the contact address inconsistency, now item 5 under Fixed.

Still open

- FlyRank graduate badge links to the course homepage, marked `data-placeholder` in the markup, pending the real verification URL.